end of the list. If you need to efficiently add or remove elements from both ends of a list, use 'deque' (double-ended queue) from the 'collections' module. Unlike standard lists, deque provides fast O (1) performance for 'appendleft()' and 'popleft()' operations. These operations are slow (O (n)) for lists.

```
from collections import deque

dq = deque([10, 15, 20])

#Appending an element
dq.appendleft(5)
#appendleft() adds element to the left end of the list

#append()          #adds element to the right end of the list

dq.append(25)
print(dq)          # Output: deque([5, 10, 15, 20, 25])

#Popping an element
l_element = dq.popleft()
r_element = dq.pop()
print(l_element)       # Output: 5
print(r_element)       # Output: 25
print(dq)                  # Output: deque([10, 15, 20])
```

'deque' is useful in queue-based algorithms, real-time data processing, and web scraping.

Quick list initialisation: Multiplication

Python's list multiplication operator (*) provides a time and memory-efficient method of list initialisation. It is useful for creating pre-populated lists quickly and works with any data type. Memory is allocated before the list is created to improve speed.

```
#Create a list named list1 containing ten 1's.
list1 = [1] * 10
print(list1)        #Output: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

The above code is equivalent to writing '[1, 1, 1, 1,..,1]' (10 times) and is a concise method to initialise such lists.

Caution: Using it with mutable elements can lead to unexpected behaviour. An example is given here.

```
nested_list = [ [0] * 3 ] * 2
nested_list [ 0 ] [ 0 ] = 1

print(nested_list)     # Output: [[1, 0, 0], [1, 0, 0]]
```

This is due to all sublists referencing the same object.

Memory-efficient lists: The array module

For lists containing only one data type (e.g., all integers or all floats), the array module offers a memory-efficient alternative. 'array' stores data more compactly, since there is no need for additional type information for each element. But it's limited to a single data type.

When creating an array, the specific type of data that it will hold has to be mentioned. We create an array that will hold only integer data types. The *'i'* represents the data type for integers.

```
from array import array
#Create an integer array

i_array = array('i', [5,10,15,20,25 ])

print(i_array)     #Output: array('i', [ 5, 10, 15, 20, 25 ])
```

Trying to add another data type will generate a type error. Most common list operations like slicing, indexing, *len()*, *extend()*, and *append()* can be done on arrays.

```
i_array = array('i', [5,10,15,20,25])
i_array.append(50)

print(i_array)  # Output: array('i', [ 5, 10, 15, 20, 25, 50 ])
```

Operations that add a different data type element to the array cannot be carried out and will generate errors.

```
i_array.append(5.5)     #Output: Error message
```

Similarly, an array that holds only floating-point numbers can be created. The *'f'* represents the data type code for floating-point numbers.

```
from array import array

#Array of floats
f_array = array('f', [ 1.1, 1.3, 1.5, 1.7, 1.9 ])

print(f_array)

#Output: array('f', [1.100000023841858, …, 1.899999976158142])
```

When working with large datasets of homogeneous data, such as sensor readings in IoT devices, or numerical data in scientific applications, arrays are particularly useful. The memory savings offered by arrays can be substantial in such scenarios, allowing you to process and store more data without running out of memory.